Strands Agents SDK — Architecture Documentation

Comprehensive reverse-engineering of the Strands Agents SDK architecture, based on analysis of the Python reference implementation.

Documents

#	Document	Diagram	Description
1	Overview	High-Level Architecture	SDK overview, design principles, package structure
2	Event Loop	Event Loop Flow	Core execution cycle — the heart of the agent
3	Model Providers	Provider Architecture	Abstract Model interface and 13+ backends
4	Tool System	Tool Architecture	Registration, execution, MCP, hot-reload
5	Hooks & Plugins	Lifecycle Events	Event-driven extension system
6	Multi-Agent	Multi-Agent Patterns	Graph, Swarm, Agents-as-Tools, A2A
7	Session & State	Session Management	Persistence, conversation management
8	Data Flow	End-to-End Flow	Complete invocation trace
9	Type Reference	—	All interfaces and types for Go port

Diagrams

All diagrams are SVG files in the `diagrams/` folder. Convert to PNG:

```
cd diagrams && for f in *.svg; do rsvg-convert -w 1800 "$f" -o "${f%.svg}.png"; done
```

Key Architectural Insights

1. **The event loop is recursive, not iterative** — each tool execution triggers `recurse_event_loop()` which creates a new cycle. This builds a trace tree.
2. **Everything is async** — the core loop is `AsyncGenerator[TypedEvent, None]`. Sync wrappers (`__call__`) use `asyncio.run()`.
3. **Hooks control flow** — retry (model and tool), cancel, and interrupt are all driven by hook events, not hard-coded logic.
4. **Tools are the only capability** — even structured output is implemented as a hidden tool that the model must call.
5. **The model drives everything** — there is no explicit state machine. The LLM decides which tools to call and when to stop.
6. **MCP is a first-class citizen** — MCP servers are treated as tool providers, not a bolt-on.
7. **Multi-agent builds on single-agent** — Graph and Swarm both compose `Agent` instances, reusing the same event loop and hook infrastructure.

Strands Agents SDK — Architecture Overview

- **Source**: github.com/strands-agents
- **Reference Implementation**: Python SDK (`strands-agents/sdk-python`)
- **Licence**: Apache 2.0

What Is Strands Agents?

Strands Agents is an open-source, model-driven SDK for building AI agents. Rather than encoding decision trees, the developer defines tools and a system prompt; the LLM itself drives the execution loop, deciding which tools to call and when to stop.

A minimal agent is three lines:

```python
from strands import Agent
from strands_tools import calculator
agent = Agent(tools=[calculator])
agent("What is the square root of 1764?")
```

Design Principles

Principle	Description
Model-driven	The LLM decides the control flow — no explicit state machines
Model-agnostic	Pluggable provider abstraction over 13+ backends
Thin core	The SDK is a ~500-line event loop + provider/tool abstractions
Tool-first	Everything an agent can do is expressed as a tool
MCP-native	First-class Model Context Protocol support for external tool servers
Observable	Built-in OpenTelemetry tracing, metrics, and hook system
Composable	Multi-agent patterns (Graph, Swarm, A2A) built on the same primitives

High-Level Architecture

The SDK consists of seven subsystems:

1. **Agent** — The top-level orchestrator. Holds messages, model, tools, hooks, and state.
2. **Event Loop** — The core cycle: send messages to model → receive response → if tool_use, execute tools → loop.
3. **Model Providers** — Abstract `Model` interface with concrete implementations for each LLM backend.
4. **Tool System** — Registry, decorator, loaders, executors, and MCP client for tool management.
5. **Hooks & Plugins** — Event-driven extension points for every lifecycle stage.
6. **Session & State** — Conversation persistence and agent state management.
7. **Multi-Agent** — Graph, Swarm, and A2A orchestration patterns.

Repository Map

Repository	Purpose
`sdk-python`	Core Python SDK (Agent, Event Loop, Models, Tools)
`sdk-typescript`	TypeScript port of the core SDK
`tools`	60+ pre-built tools (file I/O, shell, browser, Slack, AWS, etc.)
`agent-sop`	Structured Operating Procedures — markdown workflow templates
`samples`	Example agents and use cases
`docs`	Official documentation (strandsagents.com)
`evals`	Evaluation framework for testing and benchmarking agents

Repository	Purpose
`mcp-server`	MCP documentation server for AI coding assistants
`agent-builder`	Interactive terminal demo with streaming
`devtools`	Shared CI/CD workflows

Package Structure (Python SDK)

```
strands/
├── agent/                    # Agent class, AgentResult, conversation managers
│   ├── agent.py              # Main Agent orchestrator (~800 lines)
│   ├── agent_result.py       # Result dataclass (stop_reason, message, metrics)
│   ├── base.py               # AgentBase ABC
│   ├── state.py              # AgentState container
│   └── conversation_manager/ # Sliding window, summarising, null managers
├── event_loop/               # Core execution engine
│   ├── event_loop.py         # event_loop_cycle(), recurse_event_loop()
│   ├── streaming.py          # stream_messages() — model streaming bridge
│   ├── _retry.py             # ModelRetryStrategy
│   └── _recover_*.py         # Max-tokens recovery
├── models/                   # Model provider implementations
│   ├── model.py              # Abstract Model base class
│   ├── bedrock.py            # Amazon Bedrock (default)
│   ├── anthropic.py          # Anthropic direct
│   ├── openai.py             # OpenAI
│   ├── gemini.py             # Google Gemini
│   ├── ollama.py             # Ollama (local)
│   ├── llamacpp.py           # llama.cpp (local)
│   ├── litellm.py            # LiteLLM meta-router
│   ├── sagemaker.py          # SageMaker endpoints
│   └── ...                   # mistral, writer, llamaapi, openai_responses
├── tools/                    # Tool subsystem
│   ├── tools.py              # AgentTool base, PythonAgentTool, validation
│   ├── decorator.py          # @tool decorator → DecoratedFunctionTool
│   ├── registry.py           # ToolRegistry — central tool storage
│   ├── loader.py             # Directory-based discovery and hot-reload
│   ├── watcher.py            # File watcher for hot-reload
│   ├── tool_provider.py      # ToolProvider ABC
│   ├── _caller.py            # _ToolCaller for agent.tool.X() syntax
│   ├── _validator.py         # Tool use validation
│   ├── executors/            # Sequential and Concurrent executors
│   ├── mcp/                  # MCP client, agent tool adapter, instrumentation
│   └── structured_output/    # Pydantic output enforcement via tool calls
├── multiagent/               # Multi-agent orchestration
│   ├── base.py               # MultiAgentBase, MultiAgentResult, NodeResult
│   ├── graph.py              # GraphBuilder — DAG-based execution
│   ├── swarm.py              # Swarm — collaborative self-organising teams
│   └── a2a/                  # Agent-to-Agent protocol (server + executor)
├── hooks/                    # Hook event system
│   ├── events.py             # All hook event dataclasses
│   └── registry.py           # HookRegistry, HookProvider
├── plugins/                  # Plugin system
│   ├── plugin.py             # Plugin ABC
│   ├── decorator.py          # @plugin decorator
│   └── registry.py           # PluginRegistry
├── session/                  # Session persistence
│   ├── session_manager.py    # SessionManager ABC
```

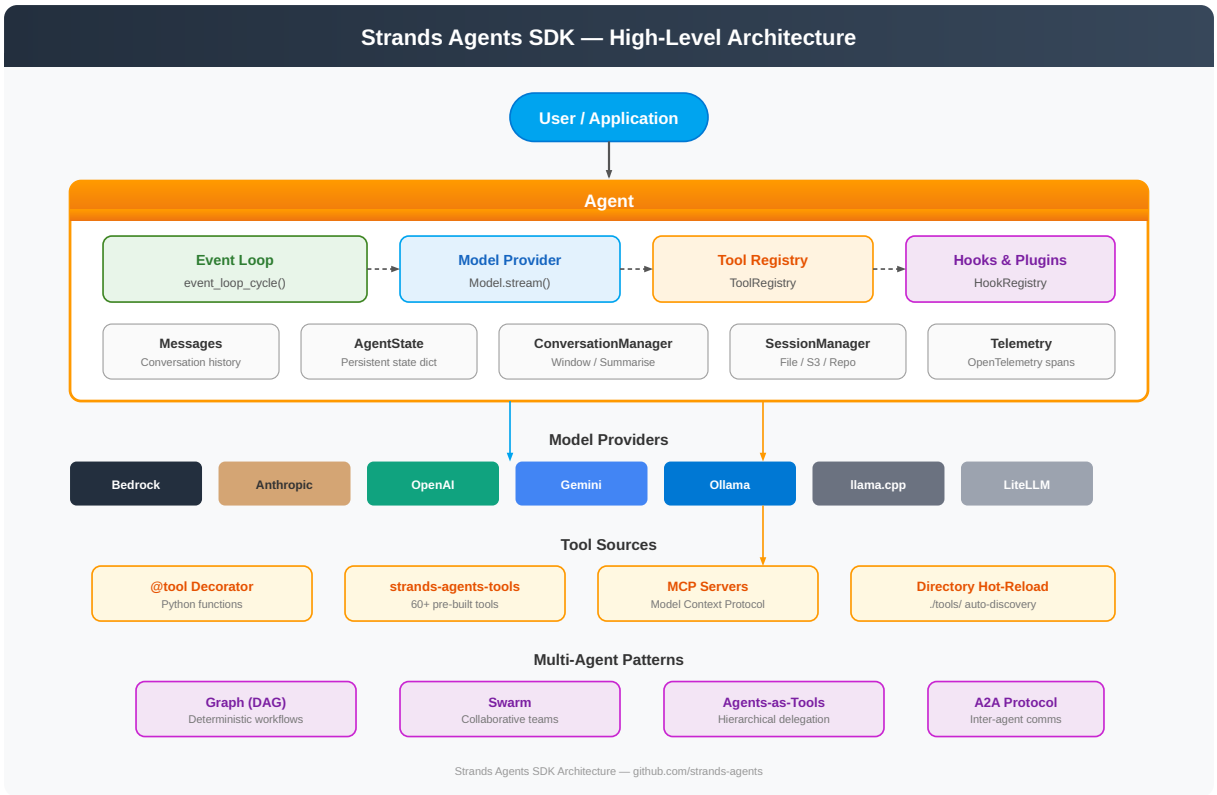

Figure 1: High-Level Architecture

```
│   ├── file_session_manager.py
│   ├── s3_session_manager.py
│   └── repository_session_manager.py
├── handlers/                # Callback handlers for streaming
├── telemetry/               # OpenTelemetry integration
│   ├── tracer.py            # Span management
│   ├── metrics.py           # Trace tree and metrics collection
│   └── config.py            # Telemetry configuration
├── types/                   # Shared type definitions
│   ├── content.py           # Message, Messages, ContentBlock
│   ├── tools.py             # ToolSpec, ToolUse, ToolResult, AgentTool
│   ├── streaming.py         # StreamEvent, StopReason
│   ├── exceptions.py        # All custom exceptions
│   └── ...                  # events, guardrails, media, etc.
└── experimental/            # Unstable features
    ├── bidi/                # Bidirectional streaming (voice/audio)
    └── steering/            # LLM-based steering and context providers
```

Event Loop — Core Execution Engine

Source: `strands/event_loop/event_loop.py`

Overview

The event loop is the heart of the Strands Agents SDK. It implements a recursive, model-driven execution cycle where the LLM decides the control flow by choosing when to call tools and when to stop.

The entire loop is **asynchronous** — implemented as `AsyncGenerator[TypedEvent, None]` yielding typed events to callers.

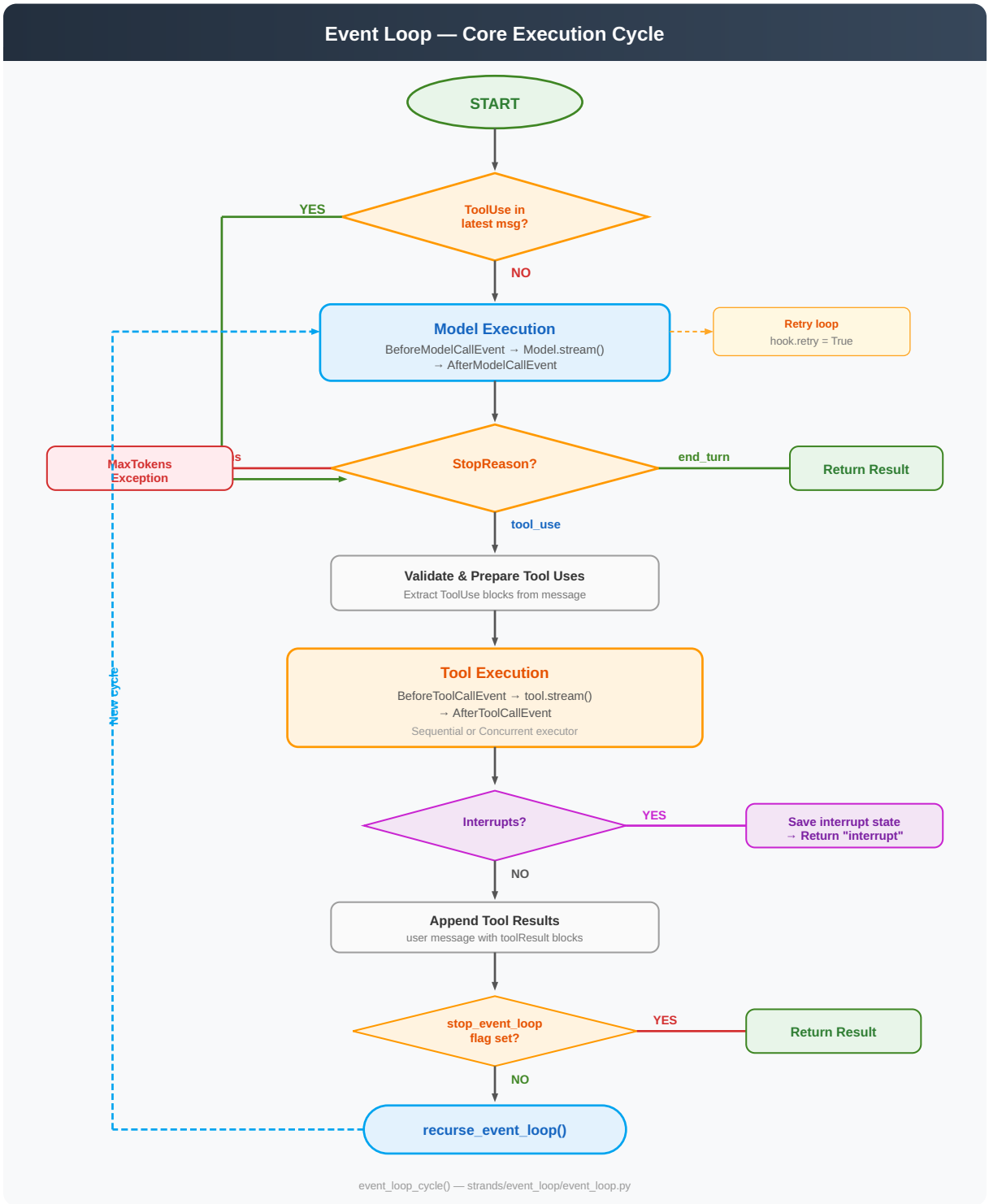

Figure 2: Event Loop

Entry Points

The call chain from the user-facing API down to the event loop is:

```
Agent.__call__(prompt)
  → Agent.invoke_async(prompt)
    → Agent.stream_async(prompt)
      → Agent._run_loop(messages)
        → Agent._execute_event_loop_cycle(invocation_state)
          → event_loop_cycle(agent, invocation_state)
```

`event_loop_cycle()` — The Core Function

A single cycle processes one "conversation turn":

Step 1: Initialisation

- Generate a unique `event_loop_cycle_id` (UUID4)
- Initialise `request_state` if not present
- Start metrics collection and OpenTelemetry tracing span
- Yield `StartEvent()` and `StartEventLoopEvent()`

Step 2: Check for Existing Tool Uses

Before calling the model, the loop checks three conditions:

Condition	Action
`agent._interrupt_state.activated`	Resume from interrupt — skip model, set `stop_reason = "tool_use"`
`_has_tool_use_in_latest_message(messages)`	Tool results already pending — skip model
Neither	Proceed to model execution

Step 3: Model Execution (`_handle_model_execution`)

- Invoke `BeforeModelCallEvent` hook
- Call `stream_messages(model, system_prompt, messages, tool_specs)`
- The model provider streams response chunks as `StreamEvent`s
- Invoke `AfterModelCallEvent` hook
- **Retry loop**: If `after_event.retry == True` (set by hooks, e.g. `ModelRetryStrategy` for throttling), discard the response and re-invoke the model
- On success: append the response message to `agent.messages`, update usage/metrics

Step 4: Process Stop Reason

`stop_reason`	Behaviour
`"end_turn"`	Model is done — yield `EventLoopStopEvent` and return
`"max_tokens"`	Raise `MaxTokensReachedException`
`"tool_use"`	Proceed to tool execution (Step 5)

Step 5: Tool Execution (`_handle_tool_execution`)

1. **Validate**: Extract `ToolUse` blocks from the message, validate structure
2. **Resume interrupts**: If resuming from interrupt, merge existing `tool_results` and filter already-completed tool uses
3. **Execute**: Call `agent.tool_executor._execute()` (Sequential or Concurrent)
4. **Check interrupts**: If any `ToolInterruptEvent` was yielded, save interrupt state and return `"interrupt"` stop reason

5. **Structured output**: If enabled, check if the structured output tool was invoked and extract the result
6. **Append results**: Build a `user` message containing `toolResult` blocks and append to `agent.messages`
7. **Check stop flag**: If `request_state["stop_event_loop"]` is set, return
8. **Recurse**: Otherwise, call `recurse_event_loop()` to start a new cycle

Step 6: Structured Output Forcing

If `structured_output_model` is enabled and the model returned `"end_turn"` without invoking the output tool: - Append a user message prompting for structured output - Set `forced_mode` on the structured output context - Recurse to get the model to produce the structured output - If it fails again, raise `StructuredOutputException`

Recursion Model

The event loop is **recursive, not iterative**. After tool execution, `recurse_event_loop()` creates a child `Trace` and calls `event_loop_cycle()` again. This creates a trace tree:

```
event_loop_cycle (cycle 1)
  ├── stream_messages (model call)
  ├── Tool: calculator
  └── Recursive call
      └── event_loop_cycle (cycle 2)
          ├── stream_messages (model call)
          └── EventLoopStopEvent (end_turn)
```

Error Handling

Exception	Handling
`ModelThrottledException`	Caught by `AfterModelCallEvent` → `ModelRetryStrategy` sets `retry = True`
`ContextWindowOverflowException`	Bubbled up to `Agent._execute_event_loop_cycle()` which calls `conversation_manager.reduce_context()` and retries
`MaxTokensReachedException`	Bubbled up directly
`EventLoopException`	Wraps any other exception with the current `request_state`
Any other `Exception`	Yields `ForceStopEvent`, then wraps in `EventLoopException`

Retry Strategy

The `ModelRetryStrategy` (default implementation) uses exponential backoff: - `MAX_ATTEMPTS = 6` - `INITIAL_DELAY = 4` seconds - `MAX_DELAY = 240` seconds (4 minutes)

It registers as a hook on `AfterModelCallEvent` and sets `event.retry = True` to trigger re-invocation.

Key Types

```
StopReason = "end_turn" | "tool_use" | "max_tokens" | "interrupt"

StreamEvent = dict with "stop" key containing:
  (stop_reason, message, usage, metrics)

TypedEvent = Union[
```

```
  StartEvent, StartEventLoopEvent, ModelMessageEvent,
  ToolStreamEvent, ToolResultEvent, ToolInterruptEvent,
  ToolCancelEvent, ForceStopEvent, StructuredOutputEvent,
  EventLoopStopEvent, ModelStopReason
]
```

Model Provider Architecture

Source: `strands/models/`

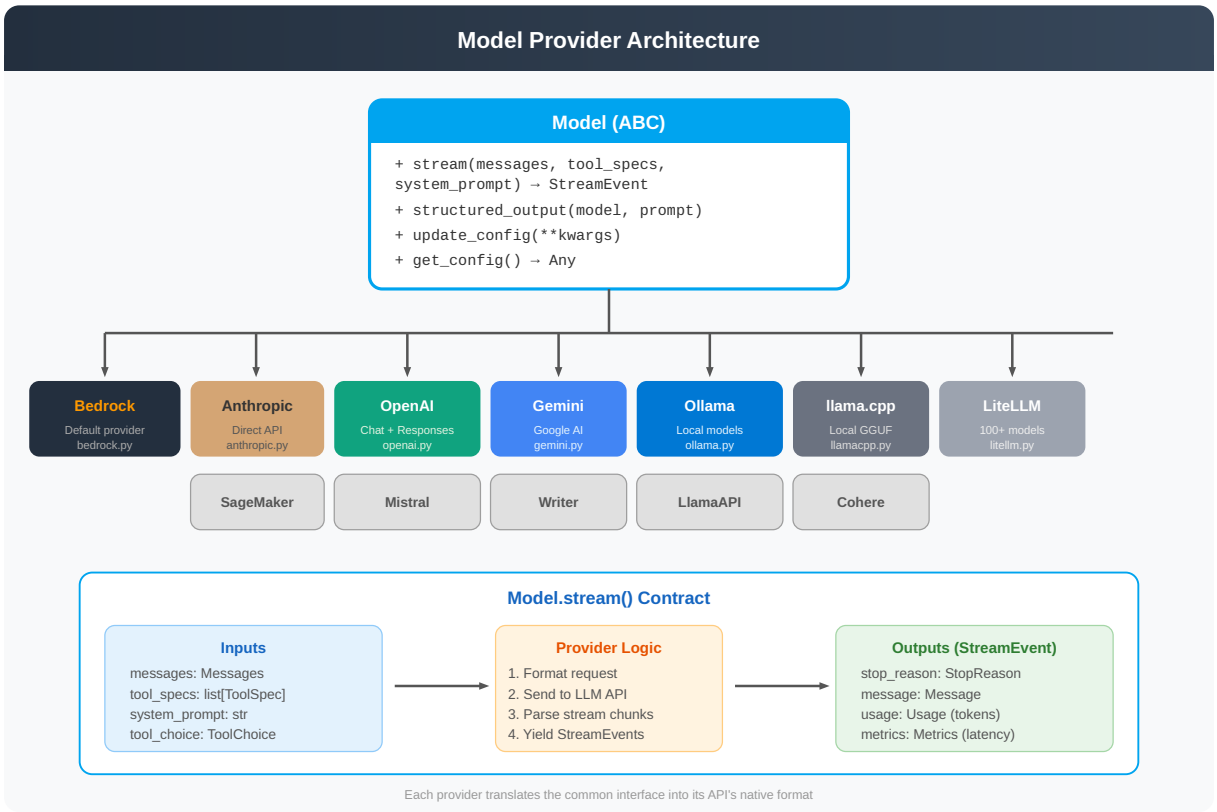

Figure 3: Model Provider Architecture

Overview

The model provider layer is a clean abstraction that isolates the agent loop from LLM-specific API details. Every provider implements the same `Model` ABC, so swapping backends is a one-line configuration change.

Abstract Interface: `Model`

```python
class Model(ABC):
    def update_config(self, **model_config: Any) -> None: ...
    def get_config(self) -> Any: ...

    def stream(
        self,
        messages: Messages,
        tool_specs: list[ToolSpec] | None = None,
        system_prompt: str | None = None,
        *,
        tool_choice: ToolChoice | None = None,
        system_prompt_content: list[SystemContentBlock] | None = None,
```

```
        invocation_state: dict[str, Any] | None = None,
    ) -> AsyncIterable[StreamEvent]: ...

    def structured_output(
        self,
        output_model: type[T],
        prompt: Messages,
        system_prompt: str | None = None,
    ) -> AsyncGenerator[dict[str, T | Any], None]: ...
```

`stream()` — The Core Method

Each provider must implement `stream()` which:

1. **Formats** the request — converts the common `Messages`, `ToolSpec`, and `SystemContentBlock` types into the provider's native API format
2. **Sends** the request to the LLM API (streaming)
3. **Yields** normalised `StreamEvent` chunks back to the event loop

The last event yielded contains a `"stop"` key with: - `stop_reason: StopReason` — why the model stopped (`end_turn`, `tool_use`, `max_tokens`) - `message: Message` — the full assistant message with text and/or tool use blocks - `usage: Usage` — token counts (input, output, cache read/write) - `metrics: Metrics` — latency in milliseconds

`structured_output()` — Typed Responses

Processes prompts and returns Pydantic model instances. Implementations typically: 1. Add the output schema to the prompt 2. Call the model 3. Parse the response into the Pydantic model 4. Raise `ValidationException` if parsing fails

Supported Providers

Provider	File	Default Model	Notes
Amazon Bedrock	`bedrock.py`	`us.anthropic.claude-sonnet-4-20250514`	SDK default, uses boto3
Anthropic	`anthropic.py`	—	Direct Anthropic API
OpenAI	`openai.py`	—	Chat completions
OpenAI Responses	`openai_responses.py`		Responses API variant
Google Gemini	`gemini.py`	—	Google AI Studio
Ollama	`ollama.py`	—	Local models via Ollama
llama.cpp	`llamacpp.py`	—	Direct GGUF inference
LlamaAPI	`llamaapi.py`	—	Hosted Llama models
Mistral	`mistral.py`	—	Mistral AI API
Writer	`writer.py`	—	Writer API
SageMaker	`sagemaker.py`	—	Custom SageMaker endpoints
LiteLLM	`litellm.py`	—	Meta-router for 100+ providers
Cohere	(community)	—	Via community contributions

Provider Implementation Pattern

Every concrete provider follows the same structure:

```
1. __init__(client_args, model_id, params)
   → Create API client with credentials
   → Store model configuration

2. stream(messages, tool_specs, system_prompt)
   → format_request(messages, tool_specs, system_prompt)
     → Convert Messages to provider's message format
     → Convert ToolSpec to provider's function/tool format
     → Build the API request body
   → send_request(request)
     → Call provider's streaming API
   → process_stream(response)
     → Parse each chunk into StreamEvent
     → Build up the full Message from chunks
     → Yield final stop event with (stop_reason, message, usage, metrics)
```

Key Types

```python
# Common message format
Message = TypedDict("Message", {"role": str, "content": list[ContentBlock]})
Messages = list[Message]

# Tool specification sent to models
ToolSpec = TypedDict("ToolSpec", {
    "name": str,
    "description": str,
    "inputSchema": dict  # JSON Schema
})

# Token usage
Usage = TypedDict("Usage", {
    "inputTokens": int,
    "outputTokens": int,
    "totalTokens": int,
    "cacheReadInputTokens": NotRequired[int],
    "cacheWriteInputTokens": NotRequired[int],
})

# Tool choice control
ToolChoice = TypedDict with "auto", "any", or "tool" key
```

Prompt Caching

The `CacheConfig` dataclass enables prompt caching: - `strategy: "auto"` — automatically inject `cachePoint` markers at optimal positions - Supported via `system_prompt_content: list[SystemContentBlock]` parameter

Error Handling

- `ModelThrottledException` — raised when the provider returns a rate-limit error; caught by the event loop's retry strategy
- Provider-specific errors are wrapped or re-raised as appropriate

Tool System Architecture

Source: `strands/tools/`

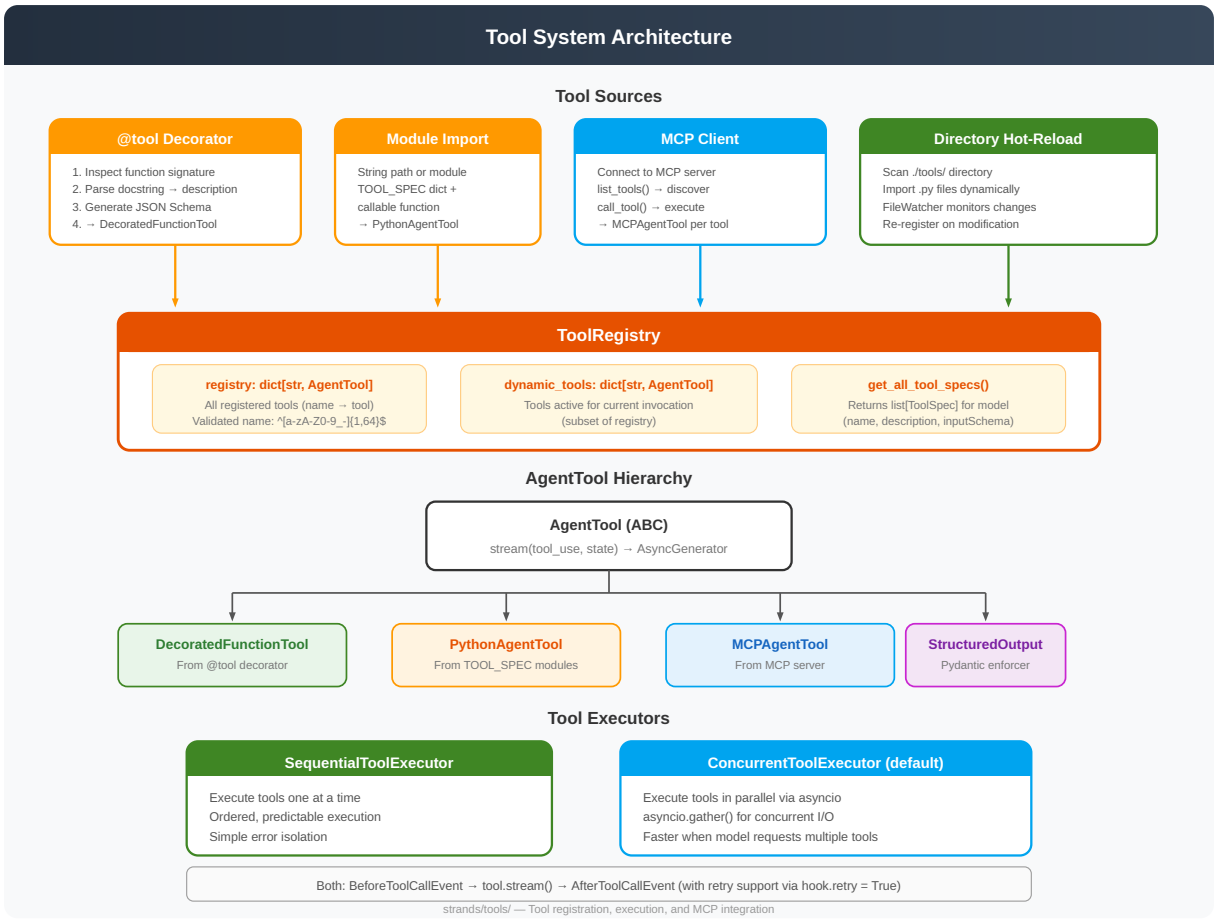

Figure 4: Tool System

Overview

The tool system is the mechanism by which agents interact with the world. Every capability — from reading files to calling APIs to delegating to sub-agents — is modelled as a tool. The system handles tool registration, validation, execution, and MCP integration.

Tool Lifecycle

`Define → Register → Discover → Validate → Execute → Result`

Defining Tools

1. `@tool` Decorator (Primary Method)

The decorator transforms a Python function into an `AgentTool`:

```python
@tool
def word_count(text: str) -> int:
    """Count words in text. The LLM sees this docstring."""
    return len(text.split())
```

Behind the scenes (`decorator.py`):

1. **`FunctionToolMetadata`** inspects the function:

- `inspect.signature()` for parameter names, types, defaults
- `get_type_hints()` for type annotations
- `docstring_parser` for description and per-parameter docs

2. **Pydantic model generation**: Creates a dynamic `BaseModel` from the function signature for input validation. Special parameters (`self`, `cls`, `agent`) are excluded.

3. **JSON Schema generation**: The Pydantic model produces a JSON Schema which becomes the tool's `inputSchema`. The schema is cleaned:

   - Remove `title`, `additionalProperties` metadata
   - Simplify `anyOf` for Optional types
   - Recursively clean nested structures

4. **`DecoratedFunctionTool`** wraps the function:

   - `__call__()` — preserves normal Python function behaviour
   - `stream()` — validates inputs via Pydantic, executes the function, wraps result in `ToolResult`
   - `__get__()` — descriptor protocol for class methods

Parameterised decoration:

```
@tool(name="custom_name", description="Override docstring", context=True)
def my_tool(ctx: ToolContext, x: int) -> str: ...
```

When `context=True`, a `ToolContext` object is injected containing the tool invocation metadata.

2. Module-Level `TOOL_SPEC` (Legacy)

Tools can be defined as Python modules with: - `TOOL_SPEC: dict` — the tool specification (name, description, inputSchema) - A callable function that accepts `(tool_use_id, input, **kwargs)`

These become `PythonAgentTool` instances.

3. MCP Client (External Tools)

The `MCPClient` connects to any Model Context Protocol server:

```
mcp_client = MCPClient(lambda: stdio_client(StdioServerParameters(
    command="uvx", args=["some-mcp-server"]
)))
agent = Agent(tools=[mcp_client])
```

Architecture (`mcp/mcp_client.py`): - Runs the MCP connection in a **background thread** with its own event loop - `start()` → initialise session, `list_tools()` to discover available tools - Each MCP tool becomes an `MCPAgentTool` in the registry - `call_tool()` proxies tool execution over the MCP protocol - Supports prompts, resources, and resource templates - Implements `ToolProvider` interface for seamless integration

4. Directory Hot-Reload

Agents can auto-discover tools from the `./tools/` directory: - `loader.py` — scans directory, imports `.py` files, registers tools - `watcher.py` — `FileWatcher` monitors for changes and re-registers on modification - Enabled via `load_tools_from_directory=True` (default)

Tool Registry

`ToolRegistry` is the central storage for all tools (`registry.py`):

Registration Flow (`process_tools`)

The registry accepts diverse input types: - String paths (`"./path/to/tool.py"`, `"strands_tools.calculator"`) - Module references - `AgentTool` instances - Dictionaries with `name` and `path` keys - Nested lists/tuples of any of the above - `ToolProvider` instances (loaded asynchronously)

Storage

- `registry: dict[str, AgentTool]` — all registered tools
- `dynamic_tools: dict[str, AgentTool]` — tools active for current invocation

Validation

- Tool names must match `^[a-zA-Z0-9_-]{1,64}$`
- Duplicate names rejected (unless hot-reload mode)
- Normalised naming prevents `my-tool` vs `my_tool` conflicts
- Required fields: `name`, `description`, `inputSchema`

Output

- `get_all_tool_specs() → list[ToolSpec]` — returns specs for the model

Tool Execution

Executor Interface (`executors/_executor.py`)

The abstract `ToolExecutor` provides:

```
class ToolExecutor(ABC):
    @abstractmethod
    def _execute(agent, tool_uses, tool_results, ...) -> AsyncGenerator[TypedEvent]: ...
```

The `_stream()` static method handles single-tool execution with: - Tool lookup from registry - `BeforeToolCallEvent` hook (can cancel, interrupt, or modify) - Tool execution via `selected_tool.stream()` - `AfterToolCallEvent` hook (can modify result or retry) - Retry loop: if `after_event.retry == True`, discard result and re-execute

Sequential Executor (`executors/sequential.py`)

Executes tools one at a time, in order. Simple and predictable.

Concurrent Executor (`executors/concurrent.py`) — Default

Executes all tool uses in parallel via `asyncio.gather()`. Faster when the model requests multiple tools simultaneously.

Structured Output

The structured output system (`structured_output/`) enforces Pydantic model schemas:

1. A hidden tool is registered whose input schema matches the Pydantic model
2. The model is instructed to call this tool with the output
3. If the model ignores the tool on `end_turn`, the system forces a retry with `tool_choice` set
4. The tool result is parsed into the Pydantic model

AgentTool Hierarchy

```
AgentTool (ABC)
  ├── tool_name: str
  ├── tool_spec: ToolSpec
  └── stream(tool_use, state) → AsyncGenerator

  ├── DecoratedFunctionTool  (@tool decorator)
  ├── PythonAgentTool        (TOOL_SPEC modules)
  ├── MCPAgentTool           (MCP server tools)
  └── StructuredOutputTool   (Pydantic enforcement)
```

`_ToolCaller` — Direct Tool Invocation

The `agent.tool` property returns a `_ToolCaller` that enables method-style access:

```
result = agent.tool.calculator(expression="2 + 2")
```

This bypasses the LLM and calls the tool directly, optionally recording it in conversation history (`record_direct_tool_call=True`).

Hooks, Plugins, and Lifecycle Events

Source: `strands/hooks/`, `strands/plugins/`

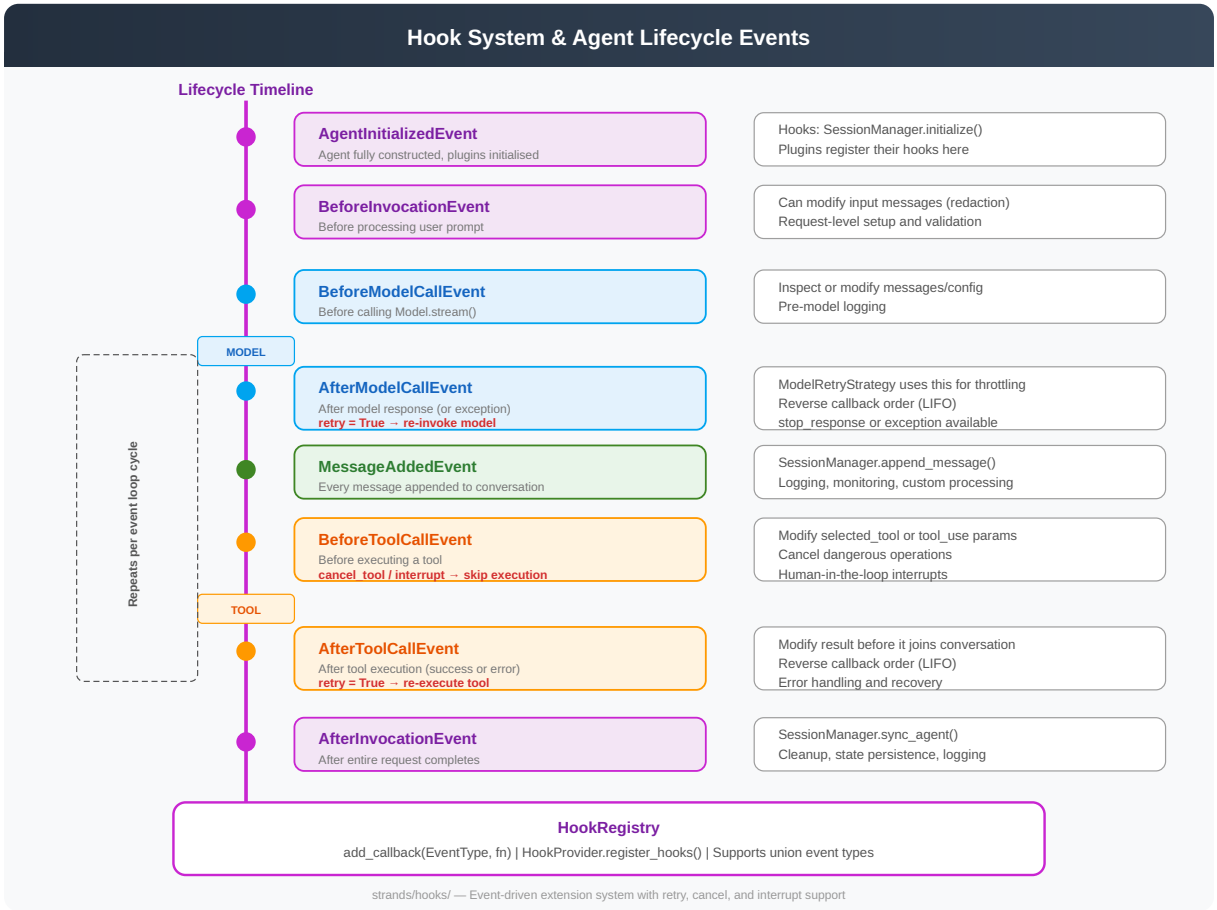

Figure 5: Hooks Lifecycle

Overview

The hook system is the primary extension mechanism for the SDK. Every significant lifecycle event fires a typed hook event, and any number of callbacks can be registered to react, modify state, or control flow (retry, cancel, interrupt).

Hook Registry

`HookRegistry` (`hooks/registry.py`) manages callback registration and invocation:

```
registry = HookRegistry()
registry.add_callback(BeforeToolCallEvent, my_callback)
```

Key Features

- **Type-safe**: Callbacks are registered against specific event types

- **Union types**: A callback can register for multiple event types at once
- **Async invocation**: `invoke_callbacks_async(event)` calls all registered callbacks
- **Ordering**: Most events use FIFO order; cleanup events (`After*`) use LIFO (reverse)

HookProvider Interface

Components that need to register hooks implement `HookProvider`:

```
class HookProvider(ABC):
    def register_hooks(self, registry: HookRegistry, **kwargs) -> None: ...
```

Built-in hook providers: - `SessionManager` — persists messages and state - `ModelRetryStrategy` — handles throttling retries - `ConversationManager` — manages context window - Plugins

Lifecycle Events

Agent Lifecycle

Event	When	Writable Fields	Reversible
`AgentInitializedEvent`	After agent construction	—	No
`BeforeInvocationEvent`	Before processing user prompt	`messages`	No
`AfterInvocationEvent`	After request completes	—	Yes (LIFO)

Model Lifecycle (per cycle)

Event	When	Writable Fields	Reversible
`BeforeModelCallEvent`	Before `Model.stream()`	—	No
`AfterModelCallEvent`	After model response	`retry`	Yes (LIFO)

Retry mechanism: Setting `after_event.retry = True` discards the current response and re-invokes the model. The `ModelRetryStrategy` uses this for exponential backoff on throttling.

Tool Lifecycle (per tool call)

Event	When	Writable Fields	Reversible
`BeforeToolCallEvent`	Before tool execution	`cancel_tool`, `selected_tool`, `tool_use`	No
`AfterToolCallEvent`	After tool execution	`result`, `retry`	Yes (LIFO)

Cancel: Setting `cancel_tool = True` (or a message string) skips execution and returns an error result.

Interrupt: Calling `event.interrupt(name)` on `BeforeToolCallEvent` pauses the agent, saving state for later resumption. This enables human-in-the-loop workflows.

Retry: Setting `after_event.retry = True` discards the result and re-executes the tool.

Message Events

Event	When	Writable Fields
`MessageAddedEvent`	Every message appended	—

Fired for: user messages, assistant responses, and tool result messages.

Multi-Agent Events

Event	When
`MultiAgentInitializedEvent`	After orchestrator initialisation
`BeforeMultiAgentInvocationEvent`	Before orchestrator execution
`AfterMultiAgentInvocationEvent`	After orchestrator completes
`BeforeNodeCallEvent`	Before individual node execution
`AfterNodeCallEvent`	After individual node completes

`BeforeNodeCallEvent` also supports `cancel_node` and `interrupt()`.

Event Properties

All events inherit from `HookEvent` / `BaseHookEvent`:

```
@dataclass
class HookEvent:
    agent: Agent  # The agent that fired the event
```

Events use `_can_write(name)` to control which fields callbacks can modify, preventing accidental corruption.

Plugin System

Plugins are higher-level extensions that bundle hook registrations (`plugins/`):

```
class Plugin(ABC):
    def register_hooks(self, registry: HookRegistry) -> None: ...
```

The `@plugin` decorator can also be used. Plugins are initialised after the agent is fully constructed, during `AgentInitializedEvent` processing.

Built-in Hook Consumers

Consumer	Hooks Used	Purpose
`SessionManager`	`AgentInitialized`, `MessageAdded`, `AfterInvocation`	Persist conversations
`ModelRetryStrategy`	`AfterModelCall`	Exponential backoff on throttle
`SlidingWindowConversationManager`	(Called by agent on overflow)	Context window management
`StructuredOutputContext`	(Internal, via event loop)	Force structured output tool

Interrupt Flow

Interrupts enable human-in-the-loop patterns:

1. Hook on `BeforeToolCallEvent` calls `event.interrupt("confirm_action")`
2. Event loop stores `_interrupt_state` with the tool use message and partial results
3. Agent returns with `stop_reason = "interrupt"` and `AgentResult.interrupts` populated
4. Application presents interrupt to user, collects response
5. On next `agent(prompt)` call, the interrupt state is restored and tool execution resumes

Multi-Agent Patterns

Source: `strands/multiagent/`

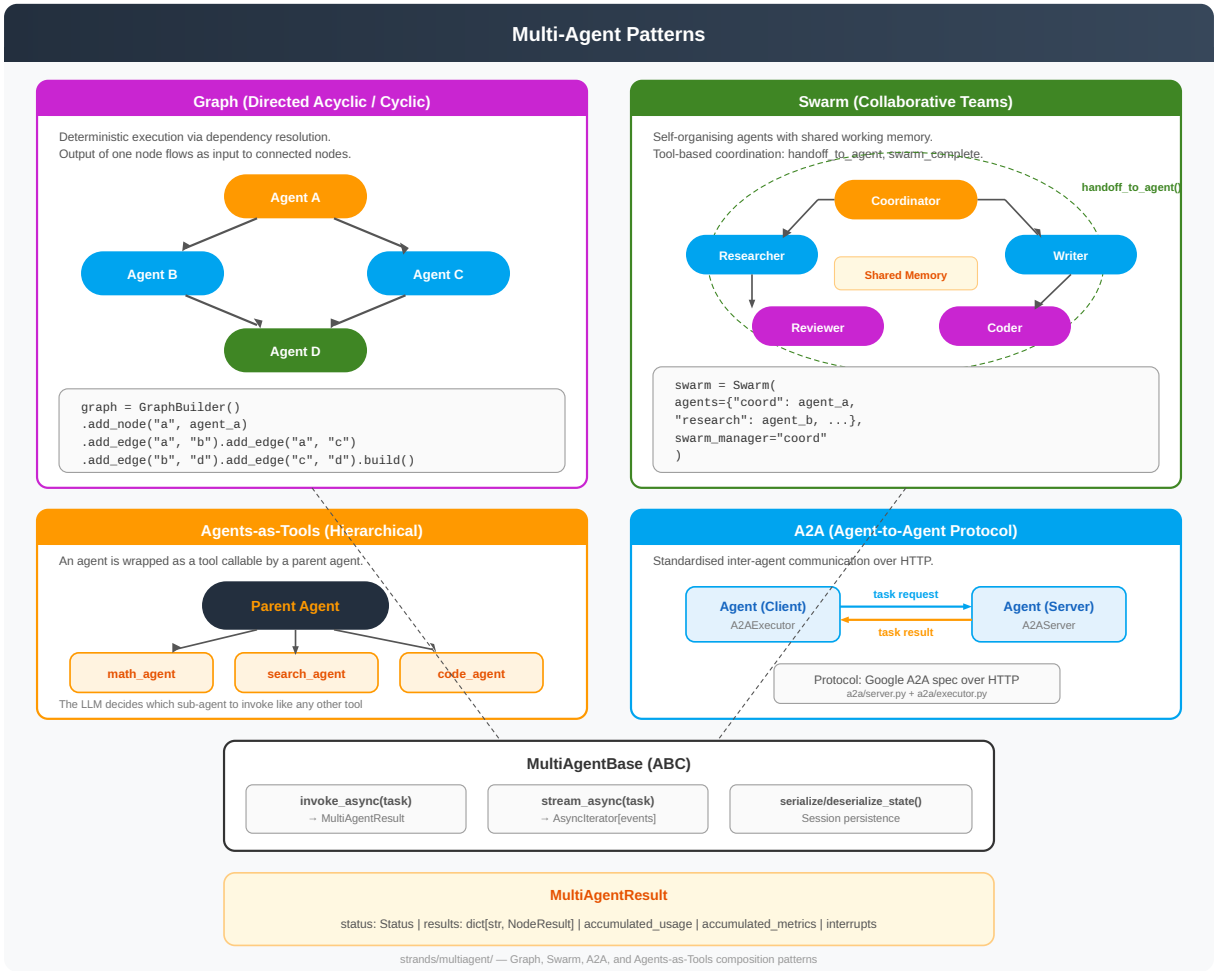

Figure 6: Multi-Agent Patterns

Overview

The multi-agent module provides four composition patterns for orchestrating multiple agents. All patterns build on a shared `MultiAgentBase` abstract class and produce `MultiAgentResult` outputs.

Base Classes

`MultiAgentBase`

```
class MultiAgentBase(ABC):
    id: str

    async def invoke_async(self, task, invocation_state=None) -> MultiAgentResult: ...
    async def stream_async(self, task, invocation_state=None) -> AsyncIterator[dict]: ...
    def __call__(self, task, invocation_state=None) -> MultiAgentResult: ...  # sync wrapper

    def serialize_state(self) -> dict: ...
    def deserialize_state(self, payload: dict) -> None: ...
```

MultiAgentResult

```python
@dataclass
class MultiAgentResult:
    status: Status              # PENDING | EXECUTING | COMPLETED | FAILED | INTERRUPTED
    results: dict[str, NodeResult]  # node_id → result
    accumulated_usage: Usage    # Total tokens across all agents
    accumulated_metrics: Metrics
    execution_count: int
    execution_time: int
    interrupts: list[Interrupt]
```

NodeResult

```python
@dataclass
class NodeResult:
    result: AgentResult | MultiAgentResult | Exception  # Supports nesting
    execution_time: int
    status: Status
    accumulated_usage: Usage
    accumulated_metrics: Metrics
    execution_count: int
    interrupts: list[Interrupt]
```

Pattern 1: Graph (DAG-Based Execution)

Source: `multiagent/graph.py`

Deterministic execution of agents based on dependency edges. Agents are nodes in a directed graph; output from one node flows as input to connected nodes.

Key Features

- **Dependency resolution**: Nodes execute when all predecessors complete
- **Parallel execution**: Independent nodes run concurrently via `asyncio.gather()`
- **Output propagation**: Each node's output is formatted and passed along edges
- **Cyclic graph support**: Feedback loops with configurable iteration limits
- **Nestable**: A `Graph` can be a node in another `Graph`
- **Conditional edges**: Edge functions can filter/transform data flowing between nodes

API

```python
graph = GraphBuilder()
graph.add_node("research", research_agent)
graph.add_node("summarise", summary_agent)
graph.add_node("review", review_agent)
graph.add_edge("research", "summarise")
graph.add_edge("summarise", "review")

result = graph.build()("Analyse quantum computing trends")
```

Node Types

- `Agent` — a standard Strands agent
- `MultiAgentBase` — a nested Graph, Swarm, or any multi-agent orchestrator
- Each node wraps its agent and manages invocation state independently

Execution Flow

1. Identify root nodes (no predecessors)
2. Execute root nodes in parallel

3. For each completed node, check which successors have all predecessors done
4. Execute ready successors
5. Repeat until all nodes complete or a node fails

Hooks

The graph fires multi-agent hooks at each stage: - `MultiAgentInitializedEvent` on build - `BeforeNodeCallEvent` / `AfterNodeCallEvent` per node - `BeforeMultiAgentInvocationEvent` / `AfterMultiAgentInvocationEvent` for the whole graph

Pattern 2: Swarm (Collaborative Teams)

Source: `multiagent/swarm.py`

Self-organising agent teams with shared working memory and tool-based coordination.

Key Features

- **Shared working memory**: All agents read/write to a common context
- **Tool-based handoffs**: Agents use `handoff_to_agent(agent_name, task)` to delegate work
- **`swarm_complete()`**: Special tool that signals the swarm task is done
- **No central controller**: Agents decide when to hand off based on LLM reasoning
- **Configurable manager**: One agent acts as the initial coordinator

API

```
swarm = Swarm(
    agents={
        "coordinator": coordinator_agent,
        "researcher": research_agent,
        "writer": writer_agent,
        "reviewer": review_agent,
    },
    swarm_manager="coordinator",
)

result = swarm("Write a technical report on AI safety")
```

Execution Flow

1. The swarm manager agent receives the initial task
2. The manager (or any active agent) calls `handoff_to_agent("researcher", "Find papers on...")`
3. The researcher agent executes with the shared context
4. The researcher may hand off to `writer`, which may hand off to `reviewer`
5. Any agent can call `swarm_complete(result)` to end the swarm
6. All agents share a working memory that accumulates context

Injected Tools

The swarm automatically injects coordination tools into each agent: - `handoff_to_agent(agent_name: str, task: str)` — delegate to another agent - `swarm_complete(result: str)` — signal task completion

Pattern 3: Agents-as-Tools (Hierarchical)

This is not a separate class but a usage pattern. An agent is wrapped as a tool:

```
@tool
def research_agent_tool(query: str) -> str:
    """Delegate research queries to the research specialist."""
    result = research_agent(query)
    return str(result)
```

```
parent = Agent(tools=[research_agent_tool, write_tool, review_tool])
parent("Write a report on AI safety with thorough research")
```

The parent agent's LLM decides when to invoke the sub-agent, just like any other tool. This creates a natural hierarchy.

Pattern 4: A2A (Agent-to-Agent Protocol)

Source: `multiagent/a2a/`

Standardised inter-agent communication over HTTP, following the Google A2A specification.

Components

Component	File	Role
`A2AServer`	`a2a/server.py`	Exposes an agent as an A2A-compatible HTTP server
`A2AExecutor`	`a2a/executor.py`	Client that sends tasks to remote A2A agents
`_converters`	`a2a/_converters.py`	Converts between Strands and A2A message formats

Usage

Server side:

```
server = A2AServer(agent=my_agent, host="0.0.0.0", port=8080)
server.start()
```

Client side:

```
executor = A2AExecutor(url="http://agent-host:8080")
result = executor("Analyse this dataset")
```

This enables distributed agent systems where agents run as separate services and communicate via a standard protocol.

Session Persistence

All multi-agent patterns support state serialisation: - `serialize_state()` → JSON-serialisable dict - `deserialize_state(payload)` → restore from dict - `SessionManager` hooks fire on `MultiAgentInitializedEvent`, `AfterNodeCallEvent`, and `AfterMultiAgentInvocationEvent`

Comparison

Feature	Graph	Swarm	Agents-as-Tools	A2A
Execution model	Deterministic DAG	Self-organising	LLM-decided	Request/response
Parallelism	Yes (independent nodes)	No (sequential handoffs)	No (tool calls)	Yes (separate services)
Shared state	Via edge outputs	Shared working memory	Via parent context	Via protocol
Nesting	Yes	Yes (Graph in Swarm)	Natural	Via composition

Feature	Graph	Swarm	Agents-as-Tools	A2A
Best for	Pipelines, workflows	Creative collaboration	Simple delegation	Distributed systems

Session & State Management

Source: `strands/session/`, `strands/agent/state.py`, `strands/agent/conversation_manager/`

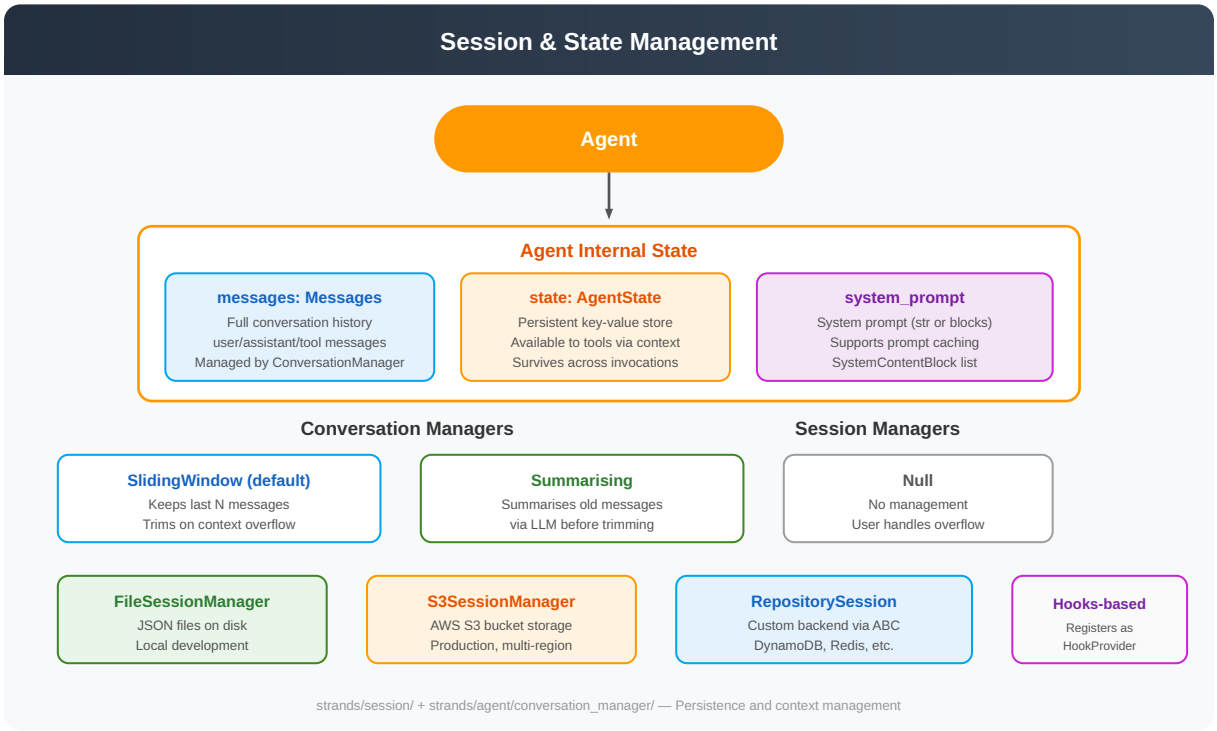

Figure 7: Session & State

Overview

The SDK manages three kinds of state: 1. **Conversation history** (`messages`) — the full message list 2. **Agent state** (`state`) — a persistent key-value store 3. **Session persistence** — serialising the above to external storage

Agent Internal State

Messages (`agent.messages: Messages`)

The conversation history is a list of `Message` dicts:

```
Message = {"role": str, "content": list[ContentBlock]}
```

Roles: `"user"`, `"assistant"`

Content blocks can be: - `{"text": str}` — plain text - `{"toolUse": ToolUse}` — model requesting a tool call - `{"toolResult": ToolResult}` — tool execution result - `{"image": ...}` — image content - And other provider-specific types

Messages are appended by the event loop and tool executors. The `MessageAddedEvent` hook fires for each addition.

AgentState (`agent.state: AgentState`)

A persistent dictionary-like container available to tools via `ToolContext`:

```
agent = Agent(state={"counter": 0})

@tool(context=True)
def increment(ctx: ToolContext):
    ctx.agent.state["counter"] += 1
```

The state survives across invocations (multiple `agent(prompt)` calls) and is persisted by the session manager.

System Prompt

Dual representation: - `agent.system_prompt: str | None` — backward-compatible string - `agent._system_prompt_content: list[SystemContentBlock]` — advanced, supports prompt caching

Conversation Managers

Conversation managers handle context window overflow by trimming or summarising old messages.

`SlidingWindowConversationManager` (Default)

Configured with a `window_size` (number of messages to keep). When the context window overflows:

1. `ContextWindowOverflowException` is caught by `_execute_event_loop_cycle()`
2. `reduce_context()` is called
3. The manager trims the oldest messages, keeping the most recent `window_size` messages
4. The event loop retries

`SummarizingConversationManager`

Before trimming, it: 1. Takes the messages that would be trimmed 2. Sends them to the LLM for summarisation 3. Replaces them with a single summary message 4. Then applies sliding window

This preserves more context at the cost of an extra LLM call.

`NullConversationManager`

No management — the user is responsible for handling overflow. Useful when the application manages context externally.

Session Managers

Session managers persist the agent's state across process restarts. They register as `HookProvider`s and react to lifecycle events.

Hook Registration

```
class SessionManager(HookProvider, ABC):
    def register_hooks(self, registry):
        registry.add_callback(AgentInitializedEvent, self.initialize)
        registry.add_callback(MessageAddedEvent, self.append_message)
        registry.add_callback(MessageAddedEvent, self.sync_agent)
        registry.add_callback(AfterInvocationEvent, self.sync_agent)
        # Multi-agent hooks
        registry.add_callback(MultiAgentInitializedEvent, self.initialize_multi_agent)
        registry.add_callback(AfterNodeCallEvent, self.sync_multi_agent)
        registry.add_callback(AfterMultiAgentInvocationEvent, self.sync_multi_agent)
```

Abstract Methods

Method	Purpose
`initialize(agent)`	Load existing session or create new one

Method	Purpose
`append_message(message, agent)`	Persist a new message
`sync_agent(agent)`	Full agent state sync
`redact_latest_message(message, agent)`	Replace last message (PII redaction)

`FileSessionManager`

- Persists to JSON files on disk
- One file per session
- Best for local development

`S3SessionManager`

- Persists to AWS S3 bucket
- Supports multi-region deployment
- Suitable for production

`RepositorySessionManager`

- Backed by abstract `SessionRepository` interface
- Implement for DynamoDB, Redis, PostgreSQL, etc.
- Most flexible option

Multi-Agent Session Support

Multi-agent orchestrators (Graph, Swarm) have additional session methods: - `initialize_multi_agent(source)` — restore orchestrator state - `sync_multi_agent(source)` — persist orchestrator state

These use `serialize_state()` / `deserialize_state()` on the `MultiAgentBase` to capture the full execution state including node results, accumulated metrics, and interrupt state.

Concurrency

The agent uses a `threading.Lock` (`_invocation_lock`) to prevent concurrent invocations: - `ConcurrentInvocationMode.THROW` (default) — raises `ConcurrencyException` - `ConcurrentInvocationMode.UNSAFE_REENTRANT` — allows concurrent access (unsafe, for advanced use)

End-to-End Data Flow

Complete Flow of a Single Invocation

This traces what happens when you call `agent("What's the weather in London?")`:

Step 1: User Input

```
result = agent("What's the weather in London?")
```

- `__call__` → `invoke_async` → `stream_async` → `_run_loop`
- The prompt string is converted to `Message(role="user", content=[{"text": "..."}])`
- Appended to `agent.messages`

Step 2: BeforeInvocationEvent

- `SessionManager.initialize()` restores previous session if available
- Guardrails or PII redaction hooks can modify `event.messages`
- Any setup/validation logic runs here

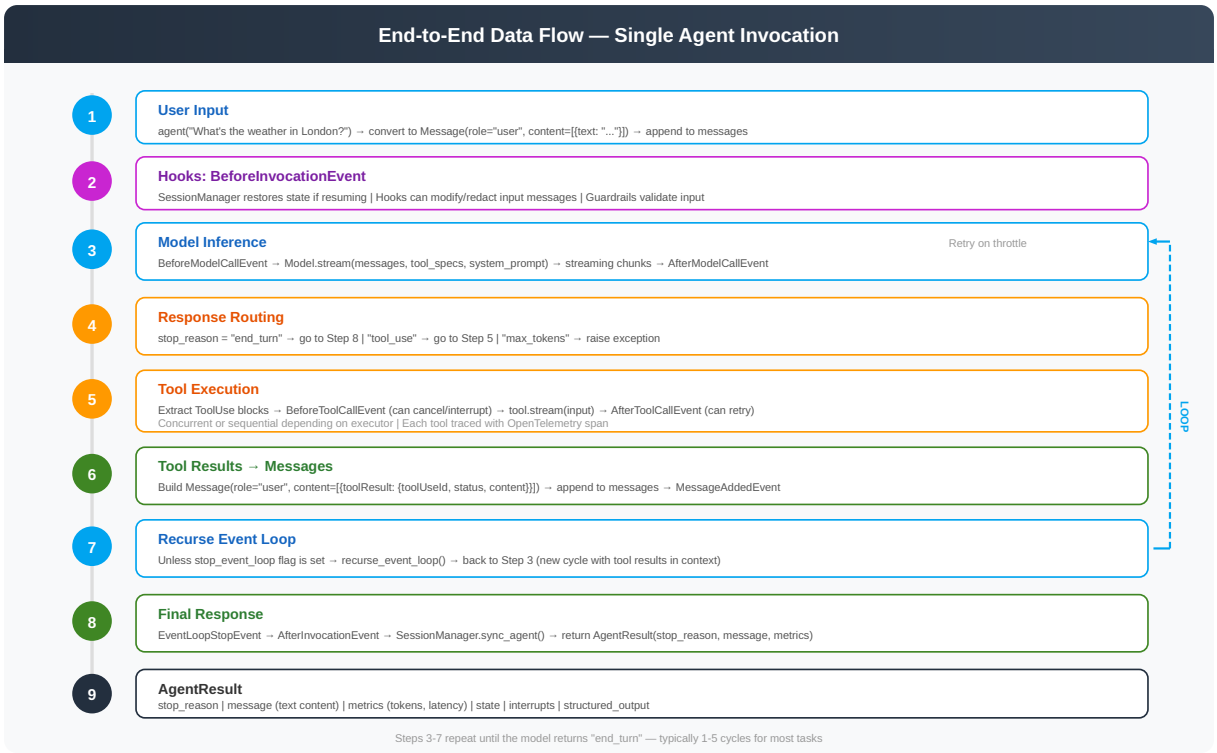

Figure 8: Data Flow

Step 3: Model Inference

- `BeforeModelCallEvent` fires
- `stream_messages()` calls `model.stream(messages, tool_specs, system_prompt)`
- The model sees the full conversation + available tools
- Streaming chunks are yielded back as events
- `AfterModelCallEvent` fires — can trigger retry on throttle

Step 4: Response Routing

The model's `stop_reason` determines the next step: - `"end_turn"` → the model has a final answer, skip to Step 8 - `"tool_use"` → the model wants to call tool(s), proceed to Step 5 - `"max_tokens"` → response was truncated, raise `MaxTokensReachedException`

Step 5: Tool Execution

For each `ToolUse` block in the assistant's message: 1. `BeforeToolCallEvent` fires — hooks can cancel, interrupt, or modify 2. The tool's `stream()` method executes with the provided inputs 3. `AfterToolCallEvent` fires — hooks can modify the result or retry

Multiple tools may execute concurrently (default) or sequentially.

Step 6: Tool Results → Messages

- Each tool result is wrapped as `{"toolResult": {"toolUseId": "...", "status": "success", "content": [...]}}`
- A single `Message(role="user", content=[...all tool results...])` is built
- Appended to `agent.messages`
- `MessageAddedEvent` fires → session manager persists

Step 7: Recurse

Unless `request_state["stop_event_loop"]` is set, the event loop recurses: - `recurse_event_loop()` starts a new cycle - The model now sees the tool results in the conversation - Back to Step 3

Step 8: Final Response

- `EventLoopStopEvent` is yielded with the final message
- `AfterInvocationEvent` fires → session manager syncs final state
- Metrics are finalized

Step 9: AgentResult Returned

```
AgentResult(
    stop_reason="end_turn",
    message={"role": "assistant", "content": [{"text": "It's currently 12°C..."}]},
    metrics=EventLoopMetrics(...),
    state={...},
    interrupts=None,
    structured_output=None,
)
```

Typical Cycle Count

Scenario	Cycles
Simple Q&A (no tools)	1
Single tool call	2 (model → tool → model)
Multi-step reasoning	3-5
Complex research task	5-10+

Message History Growth

After a 2-cycle invocation with one tool call:

```
messages = [
  {"role": "user", "content": [{"text": "What's the weather?"}]},           # User input
 {"role": "assistant", "content": [{"toolUse": {"name": "weather", ...}}]}, # Model requests tool
  {"role": "user", "content": [{"toolResult": {"status": "success", ...}}]}, # Tool result
  {"role": "assistant", "content": [{"text": "It's 12°C in London..."}]},    # Final answer
]
```

Streaming Events

Callers using `stream_async()` receive a stream of typed events throughout execution:

```
async for event in agent.stream_async("What's the weather?"):
    if "data" in event:           # Text chunk from model
        print(event["data"], end="")
    elif "tool_use" in event:     # Tool being called
        print(f"Calling {event['tool_use']['name']}...")
    elif "tool_result" in event:  # Tool completed
        print(f"Got result")
```

This enables real-time UIs that show the agent's progress as it works.

Type Reference — Key Interfaces for Go Port

This document catalogues every significant type/interface in the SDK as a reference for implementing a Go port.

Core Agent Types

Agent

```
Agent {
```

```
    model: Model
    messages: Messages
    system_prompt: str | None
    system_prompt_content: list[SystemContentBlock]
    state: AgentState
    tool_registry: ToolRegistry
    tool_executor: ToolExecutor
    hooks: HookRegistry
    callback_handler: CallbackHandler
    conversation_manager: ConversationManager
    session_manager: SessionManager | None
    retry_strategy: ModelRetryStrategy
    event_loop_metrics: EventLoopMetrics
    plugins: list[Plugin]
    trace_span: Span
    trace_attributes: dict[str, AttributeValue]
    interrupt_state: InterruptState
    invocation_lock: Lock
    concurrent_invocation_mode: THROW | UNSAFE_REENTRANT

    __call__(prompt) -> AgentResult
    invoke_async(prompt) -> AgentResult
    stream_async(prompt) -> AsyncIterator[TypedEvent]
    structured_output(prompt, model) -> T
    tool -> ToolCaller
    tool_names -> list[str]
    cleanup()
}
```

AgentResult

```
AgentResult {
    stop_reason: StopReason
    message: Message
    metrics: EventLoopMetrics
    state: Any
    interrupts: list[Interrupt] | None
    structured_output: BaseModel | None

    to_dict() -> dict
    from_dict(dict) -> AgentResult
}
```

AgentState

```
AgentState = dict[str, Any]  // persistent key-value store
```

Message Types

Message

```
Message {
    role: "user" | "assistant"
    content: list[ContentBlock]
}
```

ContentBlock (union)

```
ContentBlock =
    | {"text": str}
```

```
    | {"toolUse": ToolUse}
    | {"toolResult": ToolResult}
    | {"image": ImageBlock}
    | {"document": DocumentBlock}
    | {"guardContent": GuardContentBlock}
    | {"cachePoint": CachePointBlock}
```

SystemContentBlock

```
SystemContentBlock =
    | {"text": str}
    | {"guardContent": GuardContentBlock}
    | {"cachePoint": CachePointBlock}
```

Tool Types

ToolSpec

```
ToolSpec {
    name: str           // ^[a-zA-Z0-9_-]{1,64}$
    description: str
    inputSchema: {
        json: JSONSchema
    }
}
```

ToolUse

```
ToolUse {
    toolUseId: str
    name: str
    input: dict[str, Any]
}
```

ToolResult

```
ToolResult {
    toolUseId: str
    status: "success" | "error"
    content: list[ToolResultContent]
}
```

ToolResultContent (union)

```
ToolResultContent =
    | {"text": str}
    | {"image": ImageBlock}
    | {"json": dict}
    | {"document": DocumentBlock}
```

ToolChoice (union)

```
ToolChoice =
    | {"auto": ToolChoiceAuto{}}
    | {"any": ToolChoiceAny{}}
    | {"tool": ToolChoiceTool{name: str}}
```

AgentTool (interface)

```
AgentTool {
    tool_name: str
```

```
    tool_spec: ToolSpec
    stream(tool_use: ToolUse, state: dict) -> AsyncGenerator[event]
}
```

Model Types

Model (interface)

```
Model {
    update_config(**kwargs)
    get_config() -> Any
    stream(messages, tool_specs, system_prompt, ...) -> AsyncIterable[StreamEvent]
    structured_output(output_model, prompt, system_prompt) -> AsyncGenerator
}
```

StreamEvent

```
StreamEvent = dict with eventual "stop" key:
    stop: (StopReason, Message, Usage, Metrics)
```

StopReason

```
StopReason = "end_turn" | "tool_use" | "max_tokens" | "interrupt"
```

Usage

```
Usage {
    inputTokens: int
    outputTokens: int
    totalTokens: int
    cacheReadInputTokens?: int
    cacheWriteInputTokens?: int
}
```

Metrics

```
Metrics {
    latencyMs: int
}
```

CacheConfig

```
CacheConfig {
    strategy: "auto"
}
```

Hook Types

HookEvent (base)

```
HookEvent {
    agent: Agent
}
```

Event Types

```
AgentInitializedEvent {}
BeforeInvocationEvent { invocation_state, messages (writable) }
AfterInvocationEvent { invocation_state, result }  // reverse order
MessageAddedEvent { message }
BeforeModelCallEvent { invocation_state }
```

```
AfterModelCallEvent { invocation_state, stop_response?, exception?, retry (writable) }  // reverse
BeforeToolCallEvent { selected_tool (writable), tool_use (writable), invocation_state, cancel_tool (writab
AfterToolCallEvent { selected_tool, tool_use, invocation_state, result (writable), exception?, retry (writ
```

Multi-Agent Events (base: BaseHookEvent, no agent field)

```
MultiAgentInitializedEvent { source: MultiAgentBase, invocation_state }
BeforeMultiAgentInvocationEvent { source, invocation_state }
AfterMultiAgentInvocationEvent { source, invocation_state }  // reverse
BeforeNodeCallEvent { source, node_id, invocation_state, cancel_node (writable) }
AfterNodeCallEvent { source, node_id, invocation_state }  // reverse
```

Multi-Agent Types

MultiAgentBase (interface)

```
MultiAgentBase {
    id: str
    invoke_async(task, invocation_state) -> MultiAgentResult
    stream_async(task, invocation_state) -> AsyncIterator[dict]
    __call__(task, invocation_state) -> MultiAgentResult  // sync wrapper
    serialize_state() -> dict
    deserialize_state(dict)
}
```

MultiAgentResult

```
MultiAgentResult {
    status: Status
    results: dict[str, NodeResult]
    accumulated_usage: Usage
    accumulated_metrics: Metrics
    execution_count: int
    execution_time: int
    interrupts: list[Interrupt]
}
```

NodeResult

```
NodeResult {
    result: AgentResult | MultiAgentResult | Exception
    execution_time: int
    status: Status
    accumulated_usage: Usage
    accumulated_metrics: Metrics
    execution_count: int
    interrupts: list[Interrupt]
}
```

Status (enum)

```
Status = PENDING | EXECUTING | COMPLETED | FAILED | INTERRUPTED
```

Session Types

SessionManager (interface, HookProvider)

```
SessionManager {
    initialize(agent)
    append_message(message, agent)
    sync_agent(agent)
```

```
    redact_latest_message(message, agent)
    initialize_multi_agent(source)
    sync_multi_agent(source)
}
```

ConversationManager (interface)

```
ConversationManager {
    reduce_context(agent)
}
```

Interrupt Types

Interrupt

```
Interrupt {
    id: str
    name: str
    data: Any
}
```

InterruptState (internal)

```
InterruptState {
    activated: bool
    context: dict  // tool_use_message, tool_results
    activate()
    deactivate()
}
```

Telemetry Types

EventLoopMetrics

```
EventLoopMetrics {
    start_cycle(attributes) -> (start_time, Trace)
    end_cycle(start_time, trace, attributes)
    update_usage(usage)
    update_metrics(metrics)
    add_tool_usage(tool_use, duration, trace, success, message)
}
```

Trace (internal)

```
Trace {
    id: str
    name: str
    parent_id: str | None
    children: list[Trace]
    start_time: float
    end_time: float
    add_child(trace)
    add_message(message)
    end()
}
```